Pulse — Medical Research Digest

Web Development II (Web2) — Project Report

Title Page

Project title	Pulse — Medical Research Digest
Theme	Dr. Online
Student	Khaled Karhani
Course	Web Development II (Web2)
Instructor	*[fill in]*
Semester	Fall 2026
Submission date	November 2026
Repository	https://github.com/YOUR-USERNAME/pulse
Live deployment	https://pulse-xxx.vercel.app

Abstract

Pulse is a frontend web application that reframes the *Dr. Online* project theme as a medical research digest. Doctors publish plain-language summaries of recent studies in their specialty, and patients read, search, and discuss them in threaded conversations beneath each article.

The application was built with React 19, Vite 8, Tailwind CSS v4, and React Router v6. Authentication, sessions, and discussion persistence are handled client-side via React Context and `localStorage`, satisfying the course requirement for a frontend-only implementation while demonstrating realistic state-management patterns.

The project meets every item in the course rubric: six pages (including a dynamic `/studies/:id` route), responsive design for desktop and mobile, documented Git history, continuous deployment on Vercel, and a complete README with setup instructions and screenshots.

1. System Design

1.1 Overview

Pulse has one primary user journey: browse research, read a digest, discuss it. A secondary journey — registering as a doctor — is supported through the mock-auth flow so doctor comments carry a verified-specialty badge in the discussion section.

1.2 Component architecture

App (BrowserRouter) ├── AuthProvider (context) │ └── AuthModal (overlay) └── Layout ├── ScrollToTop ├── Navbar │ ├── Logo │ └── UserMenu (when authenticated) ├── — renders the active page │ ├── Home │ │ ├── Hero │ │ ├── Stats strip │ │ ├── Featured studies (StudyCard ×3) │ │ └── Value props │ ├── Studies │ │ ├── Search + category filter │ │ └── StudyCard grid │ ├── StudyDetails (/studies/:id) │ │ ├── Study body │ │ └── Discussion │ ├── About │ ├── Features │ ├── Contact (form + validation) │ └── NotFound └── Footer

1.3 Routes

Path	Component	Description
`/`	`Home`	Landing, featured studies
`/studies`	`Studies`	Filterable list of all studies
`/studies/:id`	`StudyDetails`	**Dynamic** — one study with discussion
`/features`	`Features`	Platform features
`/about`	`About`	Mission and how-it-works
`/contact`	`Contact`	Contact form
`*`	`NotFound`	404 fallback

1.4 State management

State	Owner	Persistence
Current user session	`AuthContext`	`localStorage` key `pulse.session`
Registered users	`AuthContext`	`localStorage` key `pulse.users`
Discussion comments	`discussions.js` module	`localStorage` key `pulse.discussions`
UI state (modal open, form drafts)	Local `useState`	Not persisted

React Context exposes `user`, `login()`, `register()`, `logout()`, and `openAuthModal()` globally. Any component can consume them with the `useAuth()` hook.

1.5 Data flow — posting a comment

User types in textarea → local draft state User clicks “Post comment” → validation in Discussion.jsx → addComment() in discussions.js → writes to localStorage → refreshes comments state → re-renders list

---

2. Technologies Used

Technology	Version	Role	Rationale
React	19	UI library	Modern hooks, concurrent rendering, industry standard
Vite	8	Build tool	Fast HMR, zero-config, better than CRA

Technology	Version	Role	Rationale
Tailwind CSS	v4	Styling	CSS-first config in v4 means no `tailwind.config.js`; design tokens live in `@theme`
React Router	6	Client-side routing	Standard SPA routing, outlet-based layouts
React Context	built-in	Global auth state	Lightweight; no Redux overhead needed for this scope
localStorage	browser	Persistence	Satisfies the frontend-only rubric requirement
Git + GitHub	—	Version control	Required by rubric; Conventional Commits style
Vercel	—	Hosting	Auto-deploy on every push to `main`, zero config for Vite

3. Key Code Snippets

3.1 Router setup — `src/App.jsx`

Nested layout route: all five pages render inside the shared `Layout`, and `/studies/:id` is the dynamic route.

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Layout />}>
      <Route index element={<Home />} />
      <Route path="about" element={<About />} />
      <Route path="features" element={<Features />} />
      <Route path="contact" element={<Contact />} />
      <Route path="studies" element={<Studies />} />
      <Route path="studies/:id" element={<StudyDetails />} />
      <Route path="*" element={<NotFound />} />
    </Route>
  </Routes>
</BrowserRouter>
```

3.2 Design tokens — `src/index.css` (Tailwind v4)

CSS-first config. Every token becomes a Tailwind utility automatically — `--color-accent` → `bg-accent`, `text-accent`, `border-accent`.

```
@import "tailwindcss";

@theme {
  --color-bg: #0a0e1a;
  --color-surface: #111827;
  --color-border: #1f2937;
  --color-fg: #e5e7eb;
  --color-fg-muted: #94a3b8;
```

```
  --color-accent: #22d3ee;
  --color-accent-hover: #67e8f9;
  --font-sans: "Inter", ui-sans-serif, system-ui, sans-serif;
  --font-mono: "JetBrains Mono", ui-monospace, monospace;
}
```

3.3 Mock authentication — `src/context/AuthContext.jsx`

Registration persists to `localStorage` and immediately opens a session.

```
function register({ name, email, password, role, specialty }) {
  const users = readUsers()
  if (users.find((u) => u.email.toLowerCase() === email.toLowerCase())) {
    return { ok: false, error: 'An account with that email already exists.' }
  }
  const newUser = {
    id: crypto.randomUUID(),
    name, email, password,
    role,
    specialty: role === 'doctor' ? specialty : undefined,
    createdAt: new Date().toISOString(),
  }
  writeUsers([...users, newUser])
  const { password: _, ...publicUser } = newUser
  writeSession(publicUser)
  setUser(publicUser)
  return { ok: true }
}
```

3.4 Dynamic route — `src/pages/StudyDetails.jsx`

`useParams` reads the URL segment, and `getStudyById` looks it up in seed data. A missing study renders a 404 block instead of crashing.

```
export default function StudyDetails() {
  const { id } = useParams()
  const study = getStudyById(id)

  if (!study) {
    return <NotFoundBlock />
  }

  return (
    <article>
      {/* ... study content ... */}
      <Discussion studyId={study.id} />
    </article>
```

```
    )
  }
```

3.5 Live search and filter — `src/pages/Studies.jsx`

`useMemo` memoizes the filtered list so it only recomputes when `query` or `cat` changes.

```
const filtered = useMemo(() => {
  const q = query.trim().toLowerCase()
  return studies.filter((s) => {
    const matchesCat = cat === 'All' || s.category === cat
    const matchesQuery =
      !q ||
      s.title.toLowerCase().includes(q) ||
      s.excerpt.toLowerCase().includes(q) ||
      s.tags.some((t) => t.toLowerCase().includes(q))
    return matchesCat && matchesQuery
  })
}, [query, cat])
```

3.6 Comment persistence — `src/data/discussions.js`

A tiny CRUD layer over `localStorage`. Delete requires ownership.

```
export function addComment({ studyId, user, body }) {
  const comment = {
    id: crypto.randomUUID(),
    studyId,
    authorId: user.id,
    authorName: user.name,
    authorRole: user.role,
    authorSpecialty: user.specialty,
    body: body.trim(),
    createdAt: new Date().toISOString(),
  }
  write([...read(), comment])
  return comment
}

export function deleteComment(id, userId) {
  const comments = read()
  const comment = comments.find((c) => c.id === id)
  if (!comment || comment.authorId !== userId) return false
  write(comments.filter((c) => c.id !== id))
  return true
}
```

---

4. Challenges and Solutions

Challenge	Solution
Tailwind v4 moved from JS config to CSS `@theme`	Learned the new pattern; placed all tokens in `index.css`
Study cards on Home and Studies duplicated markup	Extracted `StudyCard` component used by both
New routes opened mid-scroll	Added `ScrollToTop` component to `Layout`
Auth state needed in both `Navbar` and `StudyDetails`	React Context + `useAuth` hook
Mock sign-in accepted any text as email	Added regex validation before calling `login()` / `register()`

---

5. Deployment

The application is deployed on **Vercel** with automatic builds on every push to the `main` branch.

Deployment steps:

1. Signed into Vercel with GitHub.
2. Imported the `pulse` repository.
3. Set root directory to `my-app/` (code lives in a subfolder).
4. Vercel detected Vite automatically — no custom build configuration needed.
5. First deploy succeeded in ~45 seconds.

Every subsequent `git push origin main` triggers a rebuild and redeploy within a minute.

---

6. Known Limitations

- Accounts and comments exist only in the current browser's `localStorage`; clearing site data wipes them.
- No real email sending — the Contact form simulates success.
- Passwords are stored in plain text. This is explicitly insecure and would require a backend with hashing (bcrypt/argon2) in production.
- No email verification, password reset, or account recovery.

---

7. Conclusion

Pulse demonstrates a complete frontend application built on modern React tooling. It satisfies every rubric item: multi-page SPA with routing, responsive design, documented Git history, live deployment, and a complete README.

More importantly, the architecture — context-driven auth, memoized filters, a reusable card component, a small CRUD module for comments — would scale naturally into a production application by swapping

`localStorage` for API calls to a backend.

Appendix — Screenshots

See `docs/screenshots/` in the repository for:

- `home.png` — landing page
- `studies.png` — list with filter
- `study-detail.png` — article view with Key Findings and Discussion
- `auth-modal.png` — sign-in / register
- `mobile.png` — responsive nav